

Практическая работа по Информационным технологиям

Первая программа на ассемблере RISC-V

Для работы с языком ассемблера процессора RISC-V нам потребуется один из эмуляторов, так как сейчас подавляющее большинство персональных компьютеров оснащены процессорами с архитектурой x86-64. Предлагается использовать свободно распространяемый эмулятор Ripes. Он описан в Приложении 1 к этой работе в конце текста.

Теоретически наиболее эффективный способ программировать на компьютере – писать в программу в машинных кодах того процессора, который установлен внутри этого компьютера. Только так можно учесть все аппаратные особенности конкретного типа компьютера, создавая наиболее быстрые и компактные программы. Во времена расцвета мини-ЭВМ PDP-11 или CM-4 такое даже практиковалось, так как особенности их машинного кода располагали к этому. Но гораздо раньше программисты поняли, что заменив машинные коды на сокращения словесных описаний команд, более удобными для запоминания, можно добиться гораздо большей производительности при отладке программ. Такой язык называли **ассемблером**, а эти сокращения словесных описаний команд – **мнемониками**. Также в программах на языке ассемблера присутствуют **директивы** – прямые указания ассемблеру, **макровыводы**, которые используют возможности макросов, и **метки**.

Директивы позволяют разделить программу по секциям на статические данные (строковые и числовые константы) и исполняемый код, задать точку входа в программу (строку, с которой начинается выполнение программы), а также выполнить другие размещения данных в памяти. Для нашего примера нам потребуется две секции — .global (задает точку входа) и .text (задает секцию исполняемого кода). Это не единственное назначение директив или псевдооператоров, подробнее можно прочитать в [1].

Для начала напишем простую программу на языке ассемблера RISC-V, которая выполняет арифметические и логические действия над данными в регистрах.

```
.global _start
.text
_start:    add  t1, x0, x0 # t1 = 0 + 0
           addi t0, t1, 0x9 # additional immediate t0 = t1 + 9
           xori t1, t0, 0x3 # t1 = t0 xor 0x3 =
                           # = 0b1001 xor 0b1010 = 0b1010 = 0xa
           slli t1, t1, 2   # shift left t1 = t1 << 2 =
                           # 0b00001010 << 2 = 0b00101000
           jal  _start
```

Первая строка нашей программы на ассемблере начинается с директивы .global, поговорим о ней немного позже.

После директивы .global следует строка со словом .text. Это тоже директива, которая задаёт так называемую **секцию** исполняемого кода. Дело в том, что исполняемый файл в операционной системе в одном месте хранит машинные команды, в другом – инициализированные данные, и в третьем месте находится информация о количестве необходимой памяти для неинициализированных данных. Соответствующие части исполняемого файла

называются **секциями**. При загрузке исполняемого файла в память операционная система создаёт отдельные области памяти (так называемые **сегменты**) для машинного кода, для данных и для стека (этому сегменту никакие секции не соответствуют).

Ассемблер на основе текста нашей программы формирует сегменты для каждой из секций; мы должны наш исполняемый код поместить в одну секцию, описания областей памяти данных – в другую секцию. Соответствующие секции называются .text и .data. Секцию стека операционная система формирует самостоятельно. В нашей простой программе мы применяем только секцию .text; рассматриваемая директива указывает ассемблеру приступить к формированию этой секции. В более сложных программах нам придётся применять больше секций.

Далее в программе есть строка

```
_start:      add t1, x0, x0          # t1 = 0 + 0
```

Мнемоникой add обозначается команда сложения значения из регистра x0 с ним самим (additional), а результат сложения помещается в регистр t1. Надо сказать, что содержимое регистра x0 всегда равно нулю, так сделано специально. Поэтому в регистр t1 у нас тоже окажется записан ноль.

```
addi t0, t1, 0x9          # addition immediate t0 := t1 + 9
```

Похожей мнемоникой addi обозначается команда сложения значения из регистра t1 с непосредственным значением 9 (additional immediate), а результат сложения помещается в регистр t0.

В более общем виде: после команды размещены три параметра, которые называют операндами; для команды addi первый операнд задаёт, куда следует поместить данные, второй и третий операнды поясняют, над *какими* данными производится операция.

Слово _start – это метка, причём знак подчёркивания – часть слова. Напомним, что метка при трансляции преобразуется в адрес, по которому расположена следующая команда. Метка (адрес) нужны для того, чтобы знать, куда передать управление из какой-то другой части программы, то есть сделать сюда условный или безусловный переход. Адрес для нашего процессора будет задаваться 32-разрядным шестнадцатеричным числом, кроме того, на момент написания программы мы не можем знать этот адрес. Поэтому для обозначения адреса команды или начала какого-то сегмента памяти удобно использовать метки. После трансляции и работы редактора связей все метки будут преобразованы в адреса.

Следующая строка содержит логическую операцию XOR над битами в регистре t0 и непосредственным значением 3:

```
xori t1, t0, 0x3
```

Результат также помещается в регистр, указанный первым слева, т.е. в t1. Так как у нас в регистре t0 было загружено значение 9 или 00001001 в двоичной системе счисления, то мы получим следующее:

$t1 = t0 \text{ xor } 0x3 = 0b1001 \text{ xor } 0b0011 = 0b1010 = 0x\text{a}$.

Инструкции `slli` и `srl` выполняют операции логического сдвига влево и вправо соответственно, используя непосредственное значение (положительное число из 5 бит) в качестве количества сдвигов.

При логическом сдвиге влево (`slli`) младший значащий бит устанавливается равным нулю, а старший значащий бит отбрасывается.

После XOR в `t1` хранится `0x2` (`0b00001010`).

В результате операции

```
slli t1, t1, 0x2
```

получим

$t1 = t1 \ll 0x2 = 0b1010 \ll 2 = 0b00101000 = 0x28$.

Следующая строка содержит безусловный переход на метку `_start`:

```
jal _start
```

то есть на начало нашей программы.

Нам осталось вернуться к началу программы и рассмотреть строку

```
global _start
```

Слово `global` — это директива, которая требует от ассемблера считать некоторую метку «глобальной», то есть как бы видимой извне (если говорить строго, видимой извне объектного модуля; это понятие мы будем рассматривать позднее). В данном случае глобальной объявляется метка `_start`. Как мы уже знаем, это специальная метка, которой помечается точка входа в программу, то есть то место в программе, куда операционная система должна передать управление после загрузки программы в оперативную память. Ясно, что эта метка должна быть видна извне, что и достигается директивой `global`.

Итак, наша программа состоит из трёх частей: подготовки, цикла, начало которого отмечено меткой `_start`, арифметических и логических операций и безусловного перехода на начало.

Далее для справки приведены команды арифметических и битовых операций, собранные в таблицу 1.

Таблица 1

Мне- моника	Команда	opcode	funct3	Описание
addi	ADD immediate	0010011	0x0	rd=rs1 + imm
xori	XOR immediate	0010011	0x4	rd=rs1 ^ imm
ori	OR immediate	0010011	0x6	rd=rs1 imm
andi	AND immediate	0010011	0x7	rd=rs1 & imm
slli	Shift Left Logical Imm.	0010011	0x1	Imm(11:5)=0x00, rd=rs1<< imm(4:0)
srli	Shift Right Logical Imm.	0010011	0x5	Imm(11:5)=0x00, rd=rs1>> imm(4:0)
srai	Shift Right Arith Imm.	0010011	0x5	Imm(11:5)=0x20, rd=rs1>> imm(4:0)
slti	Set Less Than Imm.	0010011	0x2	rd=(rs1 < imm)?0:1
sltiu	Set Less Than Imm. Un- signed	0010011	0x3	rd=(rs1 < imm)?0:1

При логическом сдвиге вправо (srli) старший бит заполняется нулем, а младший бит отбрасывается [1, 2].

Инструкция srai выполняет операцию арифметического сдвига вправо.

Старший бит (msb)сохраняется.

Инструкции slti rd, rsl, imm и sltiu rd, rs1, imm сравнивают содержимое исходного регистра rs1 с непосредственным значением (imm) и устанавливают в rd значение 0, если значение исходного регистра rs1 < imm.

Задание.

Запустите эмулятор Ripes (см. Приложение 1), введите текст рассмотренного примера в окно редактора. Если вы не допустите ошибок, то, указав в правом окне «Disassembled», вы увидите дизассемблированный после трансляции код и текст вашей программы. Иначе ошибки будут подчёркнуты красным.

Запустите и исполните эту программу по шагам, наблюдая результаты в соответствующих регистрах в правом столбце.

Запустите режим Auto-clock, также наблюдая результаты в регистрах.

Напишите на ассемблере свой вариант программы, которая вместо логической операции исключающего ИЛИ использует дизъюнкцию, а вместо логического сдвига влево – логический сдвиг вправо. Измените число сдвигов.

Список литературы:

1. [Руководство по набору инструкций RISC-V, том I: Непривилегированный ISA](https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view?usp=drive_link) URL: https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view?usp=drive_link
2. RISC-V Ratified Specification URL: <https://riscv.org/specifications/ratified/>

Приложение 1

Рабочая среда Ripes

Rips - это визуальный эмулятор компьютерной архитектуры и редактор ассемблерного кода, созданный для архитектуры набора команд RISC-V.

Документация по эмулятору Ripes -

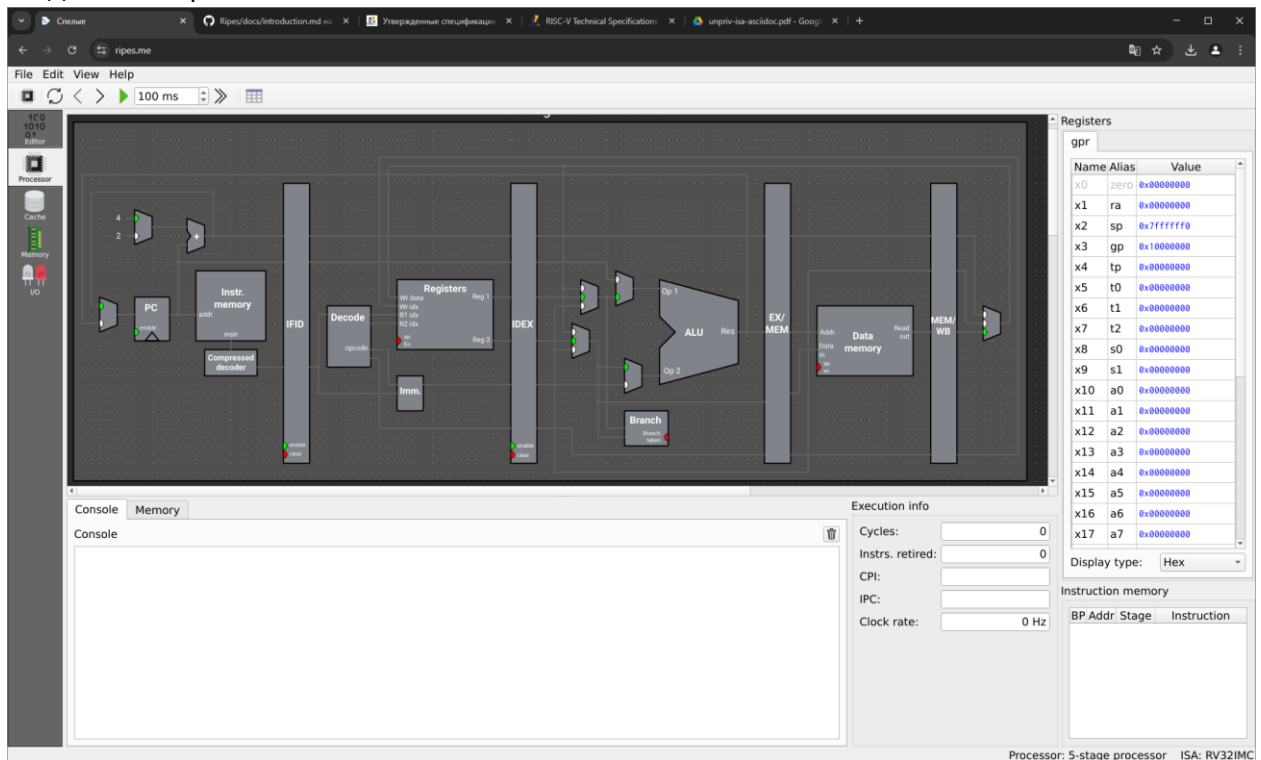
<https://github.com/mortbopet/Ripes/blob/master/docs/README.md>

Эмулятор Ripes может быть запущен тремя разными способами:

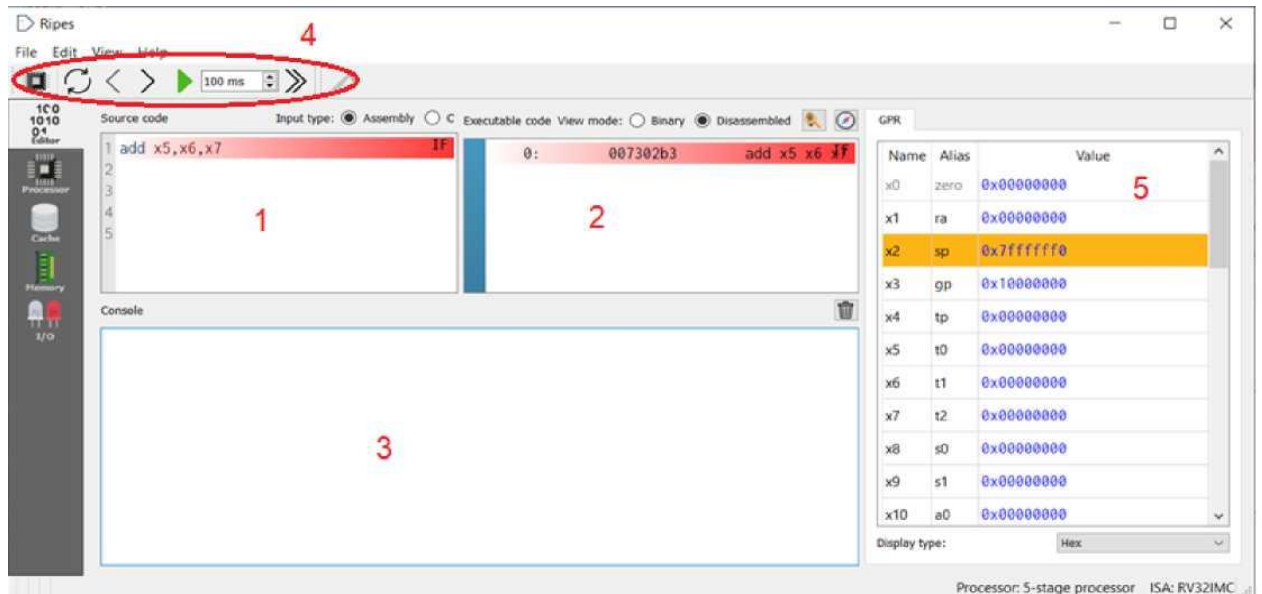
- в браузере (компилятор C не поддерживается);
- в ОС Linux;
- в ОС MS Windows.

Версия в браузере

Браузерная версия эмулятора доступна по адресу <https://ripes.me/>. Сайт может не открываться при использовании браузера Firefox. В таком случае попробуйте использовать браузер **Chrome**. Если всё получилось успешно, то должно появиться что-то похожее на данный скриншот:

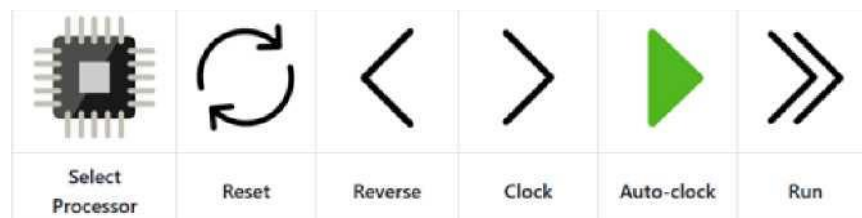


Компиляция и выполнение кода (вкладка *Editor*)



- 1 - редактор кода, в котором можно написать код на ассемблере или Си
- 2 - дизассемблированный вид написанной программы
- 3 - вывод запущенной программы
- 4 - панель инструментов
- 5 - регистровый файл

Панель инструментов



- **Select Processor:** открывает диалоговое окно выбора процессора.
- **Reset:** сбрасывает процессор, устанавливая счетчик программ на точку входа текущей программы, и сбрасывает память эмулятора.
- **Reverse:** отмена такта.
- **Clock:** синхронизирует все элементы памяти в схеме и обновляет состояние схемы.
- **Auto-clock:** синхронизирует схему с заданной частотой, заданной интервалом авто-тактирования. Автоматическая синхронизация прекратится, как только будет достигнута точка останова.
- **Run:** запускает эмулятор без выполнения обновлений графического интерфейса, чтобы сделать это как можно быстрее. Выполнение остановится после достижения точ-

ки останова или выполнения вызова `exit` `ecall`.

Исполняемый файл на Linux

Нужно скачать бинарный файл `.ApplImage` из релизов. Прямая ссылка на версию v2.2.6: https://github.com/mortbopet/Ripes/releases/download/v2.2.6/Ripes-v2.2.6-linux-x86_64.ApplImage

Release build (v.2.2.6) Latest

Ripes v2.2.6

Bug fixes and minor stuff

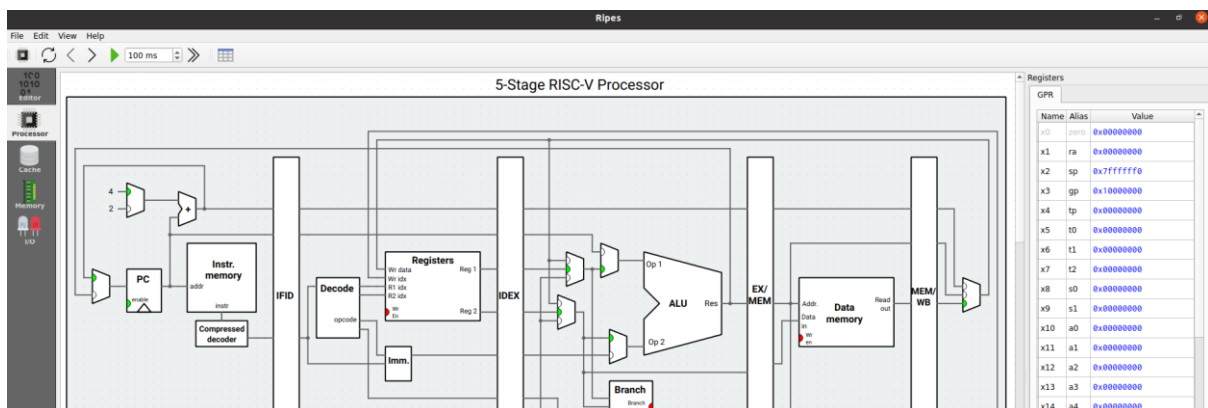
- Fixed crash when changing processor layout due to incorrect processor label IDs.
- Changed the terminology for caches, substituting "Word" for "Block". Most textbooks use the term "cache line" interchangeably with "cache line", so using "Block" for words stored in a cache line was confusing.

Assets 5

Ripes-v2.2.6-linux-x86_64.ApplImage	29.6 MB	Jan 7
Ripes-v2.2.6-mac-x86_64.zip	30.4 MB	Jan 7
Ripes-v2.2.6-win-x86_64.zip	15 MB	Jan 7
Source code (zip)		Jan 2
Source code (tar.gz)		Jan 2

6 7 4 16 people reacted

После необходимо выдать права на запуск командой `chmod +x Ripes-v2.2.6-linux-x86_64.ApplImage` . Запуск производится из терминала: `./Ripes-v2.2.6-linux-x86_64.ApplImage` . Должно открыться следующего окно:



Исполняемый файл на Windows

Нужно скачать архив для MS Windows: Ripes-v2.2.6-win-x86_64.zip (прямая ссылка: https://github.com/mortbopet/Ripes/releases/download/v2.2.6/Ripes-v2.2.6-win-x86_64.zip).

После нужно разархивировать скачанный архив и запустить файл *Ripes.exe*. Если возникла ошибка при запуске эмулятора, то, возможно, у вас отсутствует runtime библио-

Release build (v.2.2.6) Latest

Ripes v2.2.6

Bug fixes and minor stuff

- Fixed crash when changing processor layout due to incorrect processor label IDs.
- Changed the terminology for caches, substituting "Word" for "Block". Most textbooks use the term "cache" interchangeably with "cache line", so using "Block" for words stored in a cache line was confusing.

Assets 5

Ripes-v2.2.6-linux-x86_64.AppImage	29.6 MB	Jan 7
Ripes-v2.2.6-mac-x86_64.zip	30.4 MB	Jan 7
Ripes-v2.2.6-win-x86_64.zip	15 MB	Jan 7
Source code (zip)		Jan 2
Source code (tar.gz)		Jan 2

6 7 4 16 people reacted

тека. Обратитесь к соответствующему разделу в README.md (<https://github.com/mortbopet/Ripes#windows>)